

MollyTech Service Desk

Technical Documentation

Project: MollyTech Service Desk

Project Type: Full-Stack Software Engineering Project

Application: Web-Based IT Service Management System

Developer Role: Software Engineer & Full-Stack Developer — System Architecture, Development, Integration & Testing

Technology: React + Vite + Node.js + Express + MongoDB + Socket.IO

1. Project Overview

The **MollyTech Service Desk** is a full-stack web-based IT service management system designed to simulate and implement the workflow of a real-world internal IT support environment.

The system provides a centralized platform for users to submit technical support requests, track tickets, communicate with support personnel, and receive real-time updates.

Administrators and IT support personnel have dedicated interfaces for reviewing tickets, assigning support requests, communicating with users, monitoring operations, and resolving issues.

The project was intentionally developed beyond a basic CRUD application to explore **system architecture, role-based access, ticket workflows, real-time communication, notifications, authentication, database relationships, account security, and support operations.**

Intended Environment

The system can be adapted for:

- Internal corporate IT departments
- Small and medium-sized businesses
- Educational institutions
- Organizations with internal technical support teams

Developer Role

As the **Software Engineer & Full-Stack Developer**, I was responsible for:

- System architecture and design
 - Frontend development
 - Backend development
 - Database design
 - Authentication and authorization
 - Role-based access control
 - Ticket management
 - Ticket assignment
 - Real-time communication
 - Notification system
 - File and attachment handling
 - Dashboard development
 - UI/UX implementation
 - Account security functionality
 - Email notification integration
 - Testing and debugging
 - System integration
-

2. Project Motivation

MollyTech Service Desk was not developed for a specific organizational deployment.

It was created as a **self-directed software engineering project** to understand how IT service desk systems operate from both technical and operational perspectives.

The primary goal was to implement the lifecycle of an IT support request:

User → Ticket Submission → Administrative Review → Assignment → Support → Communication → Resolution

The project also provided practical experience with technologies including:

- React
- Vite
- Node.js
- Express
- MongoDB
- Socket.IO

- JWT authentication
- Role-based authorization
- Email services

The project was intentionally designed to challenge the developer beyond simple CRUD development and encourage consideration of architecture, workflows, maintainability, user experience, real-time behavior, security, system reliability, and continuous improvement.

3. Technology Stack

Technology	Purpose
React	Frontend application
Vite	Frontend development and build tooling
JavaScript	Application logic
HTML5	Web structure
CSS3	Interface styling
Node.js	Backend runtime
Express.js	Backend framework
MongoDB	NoSQL database

Socket.IO	Real-time communication
Socket.IO Client	Frontend real-time communication
JWT	Authentication
Nodemailer	Email delivery
Gmail SMTP	Development email transport
Git	Version control

4. System Features

The system provides the following major capabilities:

- User registration and approval
- Authentication and authorization
- Role-based access control
- IT support ticket creation
- Ticket assignment and reassignment
- Ticket status management
- Ticket priority management
- Ticket categorization
- Real-time messaging
- Ticket-linked conversations
- Typing indicators
- Unread message counts
- Real-time notifications
- File and image attachments
- Dashboard analytics

- Announcement management
 - Access request management
 - User account management
 - Password recovery and reset
 - Authenticated password changes
 - Password-change security notifications
 - Ticket archiving
-

5. User Roles

The system contains four primary roles:

- User
- Administrator
- IT Support
- Technician

Each role has different permissions based on its responsibilities within the service desk workflow.

5.1 User

Users can:

- View their dashboard
- Create support tickets
- View submitted tickets
- Search and filter tickets
- Edit eligible requests
- Cancel requests
- Archive tickets
- Communicate with support personnel
- Send attachments
- View notifications
- View announcements
- Manage account settings
- Change their password
- Receive password-change security notifications
- Update email and phone information

The user dashboard provides:

- Total tickets
 - Pending tickets
 - Resolved tickets
 - Messages
 - Recent tickets
 - Ticket priority
 - Ticket status
 - Quick actions
 - Announcements
-

5.2 Administrator

Administrators have the highest level of system access.

They can:

- View system-wide ticket activity
- Review submitted tickets
- Assign tickets
- Reassign tickets
- Manage users
- Approve registrations
- Manage announcements
- Access conversations
- Monitor notifications
- Review ticket records
- Manage system settings
- Change administrator password
- Receive password-change security notifications

The administrator is responsible for reviewing incoming tickets and determining the appropriate support personnel.

5.3 IT Support

IT Support personnel can manage tickets assigned to them.

They can:

- View assigned tickets
- Search tickets
- Filter tickets
- Review ticket details
- Add ticket notes
- View note history
- Communicate with users
- Resolve tickets
- Archive completed tickets
- Receive assignment notifications
- Monitor assigned ticket statistics

Their dashboard focuses on their assigned workload rather than the entire system.

5.4 Technician

Technicians can manage tickets assigned to them.

They can:

- View assigned tickets
- Search tickets
- Filter tickets
- Review ticket details
- Add ticket notes
- View note history
- Communicate with users
- Resolve tickets
- Archive completed tickets
- Receive assignment notifications
- Monitor assigned ticket statistics

Their dashboard focuses on their assigned workload and support responsibilities.

6. Ticket Management

Users create support tickets by providing:

- Subject/title

- Category
- Priority
- Email
- Phone number
- Issue description

Email and phone information are automatically populated from the user's account.

Each submitted ticket receives a unique ticket ID.

Ticket Categories

- Hardware
- Software
- Network
- Account
- Other

Priority Levels

- Low
- Medium
- High
- Urgent

Ticket Statuses

- Pending
- Resolved

The system intentionally uses **Pending** and **Resolved** as its primary states instead of separating statuses such as Open, In Progress, and Pending.

7. Ticket Workflow

The MollyTech Service Desk uses a centralized ticket workflow to manage support requests from initial submission through resolution and archival.

Ticket Processing Workflow

1. **Submit Ticket** — The user submits a support request through the service desk.

2. **Administrator Review** — The administrator receives and reviews the newly submitted ticket to determine the appropriate support action.
3. **Ticket Assignment** — The administrator assigns the ticket to the appropriate support personnel:
 - **IT Support** — Handles general technical support and troubleshooting requests.
 - **Technician** — Handles technical issues requiring more specialized assistance.
4. **Support Process** — The assigned support personnel investigates the issue, performs troubleshooting, and records relevant updates or resolution notes.
5. **User Communication** — The user can communicate with support personnel through the service desk messaging system for additional information, questions, and follow-ups.
6. **Ticket Resolution** — Once the issue has been addressed, the ticket is marked as resolved.
7. **User Notification** — The system communicates the updated ticket status to the user through the appropriate notification mechanisms.
8. **Ticket Archival** — Resolved tickets can be archived for historical reference and record keeping.

Centralized Assignment

New tickets are initially received by the administrator rather than being distributed directly to all support personnel. The administrator reviews each request and assigns it to the appropriate IT Support or Technician role.

This centralized assignment model provides controlled ticket distribution, prevents unnecessary notifications to unrelated staff members, and allows the administrator to manage workload allocation across the support team.

8. Real-Time Messaging

Real-time communication is one of the major technical components of the system.

The application uses:

Socket.IO + Node.js + Express + React

to provide real-time communication between users and support personnel.

Real-Time Features

- Real-time message delivery
- Typing indicators
- Unread message counts

- Conversation updates
- Real-time notifications
- Ticket-linked conversations

Messages are displayed without requiring users to refresh the page.

8.1 Ticket Conversations

Each ticket can have its own conversation.

Conversations contain information such as:

- Ticket ID
- Participants
- Last message
- Unread state
- Conversation status
- Timestamps

Messages are stored separately and associated with their corresponding conversation.

Users and support personnel can continue communicating even after a ticket has been resolved when further clarification or follow-up is required.

9. Notifications

The system contains a database-backed notification system with real-time updates.

User Notifications

Users can receive notifications for:

- Support assignment
- New replies
- Ticket resolution
- New announcements
- Account security events

Staff Notifications

Support personnel can receive notifications for:

- Ticket assignments
- New messages
- Ticket-related activity

Administrator Notifications

Administrators can receive notifications for:

- New tickets
- New messages
- Access requests
- System activity
- Account security events

Notifications include unread indicators and counters.

10. Dashboard and Analytics

The system provides role-specific dashboards.

Administrator Dashboard

Displays:

- Total users
- Total tickets
- Pending tickets
- Resolved tickets
- Live operations feed
- Recently registered users
- Ticket statistics

Technician Dashboard

Displays similar operational information but focuses on tickets assigned to the specific technician.

User Dashboard

Displays:

- Total tickets
- Pending tickets
- Resolved tickets
- Messages
- Recent tickets
- Ticket priority
- Ticket status
- Quick actions
- Announcements

Ticket Analytics

The system includes a graphical ticket statistics component that displays ticket activity over time.

Available filters include:

- Today
 - Week
 - Month
 - Year
-

11. Authentication and Authorization

The system implements authentication using:

- Email and password authentication
- JWT-based authentication
- Administrator account approval
- Password recovery
- Password reset
- Authenticated password changes
- Password-change security notifications

Passwords are securely hashed before being stored in the database and are never stored as plaintext.

11.1 User Approval

New users can register for the MollyTech Service Desk, but newly created accounts cannot immediately access the system. Each registration must first be reviewed and approved by an administrator.

Approval Process

1. **Registration** — The user submits the registration form and provides the required account information.
2. **Pending Account** — The newly registered account is placed in a pending state and cannot access the service desk.
3. **Administrator Review** — The administrator receives an access request notification and reviews the submitted registration.
4. **Approval** — The administrator approves the registration if the account is authorized to access the system.
5. **User Login** — Once approved, the user can log in and access the appropriate service desk features.

This approval process adds an administrative authorization layer before system access is granted, helping prevent unauthorized or unverified accounts from accessing the service desk.

11.2 Role-Based Authorization

Protected routes prevent unauthorized users from accessing role-specific functionality.

Role-based middleware controls access to protected resources.

Roles include:

- User
 - Administrator
 - IT Support
 - Technician
-

11.3 Password Management

Authenticated users can change their account password through the account settings interface.

The password-change process requires:

- Current password

- New password
- Confirmation of new password

The system validates the current password before allowing the change.

Additional validation prevents:

- Password reuse
- Passwords shorter than the required minimum length
- Mismatched password confirmation

New passwords are securely hashed using bcrypt before being stored in MongoDB.

11.4 Password-Change Security Notification

After a successful password change, MollyTech Service Desk sends a security notification to the email address associated with the account.

The notification:

- Confirms that the password was successfully changed
- Does not contain the user's password
- Uses a role-specific email template
- Provides a security warning if the change was not authorized
- Is delivered through the application's centralized email service

Current Implementation Status

Role	Template	Status
Administrator	<code>admin.password.change.html</code>	Implemented and tested
User	<code>user.password.change.html</code>	Implemented and tested
IT Support	<code>itsupport.password.change.html</code>	Pending

Technician	<code>technician.password.change.html</code>	Pending
------------	--	---------

The backend email service is structured to support role-specific password-change notifications.

Staff email templates will be finalized when the corresponding staff email accounts are configured.

12. Backend Architecture

The backend is developed using **Node.js** and **Express.js**.

The backend follows a modular structure containing:

- Config
- Controllers
- Middleware
- Models
- Routes
- Services
- Scripts
- Templates
- Utilities
- Socket handling

Major Backend Modules

- Authentication
 - Users
 - Tickets
 - Conversations
 - Messages
 - Notifications
 - Dashboard
 - Access Requests
 - Announcements
 - Email Services
-

12.1 Middleware

The application uses middleware for:

- Authentication
- Role authorization
- File uploads
- Request processing

Authentication middleware protects restricted resources while role middleware determines whether an authenticated user has permission to access a particular operation.

13. Frontend Architecture

The frontend is developed using **React and Vite**.

The frontend is organized into modular sections including:

- Admin
- User
- Authentication
- Components
- API
- Context
- Socket
- Assets
- Utilities

This structure separates application responsibilities and makes individual modules easier to maintain and debug.

14. Database Design

The system uses **MongoDB** as its primary database.

Major collections include:

- users
- tickets

- conversations
 - messages
 - notifications
 - announcements
-

14.1 Users

User documents contain information such as:

- User ID
 - Name
 - Email
 - Password
 - Role
 - Phone number
 - Account status
 - Password reset token
 - Password reset expiration
 - Created timestamp
 - Updated timestamp
-

14.2 Tickets

Ticket documents contain information such as:

- Ticket ID
 - Title
 - Description
 - Submitted by
 - Email
 - Phone number
 - Category
 - Priority
 - Status
 - Assigned personnel
 - Created timestamp
 - Updated timestamp
-

14.3 Conversations

Conversation documents contain information such as:

- Ticket ID
 - Participants
 - Last message
 - Unread state
 - Conversation status
 - Timestamps
-

14.4 Messages

Message documents contain information such as:

- Sender
 - Conversation
 - Message content
 - Attachments
 - Timestamp
-

14.5 Notifications

Notification documents contain:

- Recipient
 - Notification type
 - Related ticket
 - Message
 - Read/unread state
 - Timestamp
-

15. Database Relationships

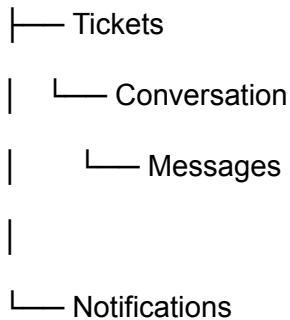
The MollyTech Service Desk uses MongoDB and ObjectId references to establish logical relationships between documents. These relationships allow the system to associate users, tickets, conversations, messages, notifications, and assigned support personnel.

Primary Relationships

1. **User → Tickets** — A user can submit multiple support tickets. Each ticket stores a reference to the user who submitted it.
2. **Ticket → Conversation** — A ticket can have an associated conversation used for ongoing communication between the user and support personnel.
3. **Conversation → Messages** — A conversation can contain multiple messages, with each message associated with the corresponding conversation.
4. **User → Notifications** — A user can have multiple notifications associated with account activity, ticket updates, and other system events.
5. **Ticket → Assigned Staff** — A ticket can be assigned to a specific IT Support staff member or Technician through an ObjectId reference.

Relationship Structure

User



Ticket



These relationships allow the system to maintain connections between support requests and their related communication, notifications, and assigned personnel while keeping each MongoDB document collection logically organized.

16. File and Attachment Handling

The messaging system supports file attachments.

Supported attachments include:

- Images
- PDF files
- Other supported files

Attachments are associated with messages and stored through the application's upload handling system.

17. Email Service

MollyTech Service Desk includes a centralized backend email service using **Nodemailer**.

The email service is responsible for delivering system-generated emails and loading reusable HTML email templates.

Current Email Functions

The system supports email notifications for:

- Account approval
- Password recovery
- Password changes
- Ticket resolution

Email Templates

Templates are stored separately from the application logic and are dynamically loaded by the email service.

Current templates include:

- account.approved.html
- reset.password.html
- admin.password.change.html
- user.password.change.html
- itsupport.password.change.html

- technician.password.change.html
- ticket.resolved.html

Role-specific password-change templates allow the system to maintain consistent branding while providing account-specific security notifications.

The email service also embeds the MollyTech logo directly into supported email messages using an inline content ID attachment.

18. Security

Implemented

- Password hashing
- JWT authentication
- Protected routes
- Role-based authorization
- Administrator approval
- Password recovery
- Password reset
- Password reuse prevention
- Minimum password-length validation
- Secure password-change workflow
- Password-change security notifications
- Passwords excluded from email notifications
- Environment variables
- Authentication middleware
- Role middleware
- Basic input validation

Sensitive configuration values such as database credentials, JWT secrets, and email credentials are maintained through environment variables rather than being directly embedded into the application source code.

19. Development and Testing

The system was tested continuously throughout development.

Testing Methods

- Browser testing
- Manual functional testing
- Database testing
- Postman
- Thunder Client
- Real-time functionality testing
- Email delivery testing
- Password-change notification testing
- Email template rendering testing

Backend functionality was tested independently before frontend integration where appropriate.

Real-time functionality was tested using multiple user roles and browser sessions to verify message delivery, notifications, ticket updates, and synchronization.

Password-change functionality was tested using Administrator and User accounts to verify:

- Current password validation
- Successful password updates
- Password reuse prevention
- Password confirmation validation
- Successful email delivery
- Correct email template selection
- MollyTech logo rendering
- Security notification content

The Administrator and User password-change email templates have been successfully implemented and tested.

IT Support and Technician password-change email templates remain pending until their corresponding staff email accounts are configured.

20. Major Development Challenges

Several significant challenges were encountered during development.

Real-Time Messaging

Ensuring messages were delivered and displayed correctly without page refreshes required careful coordination between Socket.IO, the backend, database state, and React components.

Ticket Assignment

The ticket workflow required tickets to initially reach the administrator rather than automatically appearing as actionable tickets for every support staff member.

Notifications

Notification states and unread counters needed to remain synchronized between the database and real-time frontend updates.

Conversation State

The system needed to preserve ticket conversations even after resolution so users and support personnel could continue communication when necessary.

UI Consistency

Maintaining consistent UI/UX across multiple role-specific dashboards and modules was another significant development challenge.

Email Integration

Integrating transactional email functionality required coordination between the backend email service, HTML templates, environment configuration, and frontend-triggered account actions.

Password-change notifications also required role-specific templates while maintaining consistent MollyTech branding and security messaging.

21. Debugging and Issue Resolution

Throughout development, several issues were identified and resolved, including:

- Messages not appearing correctly
- Real-time events not updating
- Notification synchronization problems
- Ticket assignment behavior
- Conversation state issues
- Database synchronization issues
- UI consistency problems
- Password-change validation
- Password reuse prevention
- Email template rendering
- Email logo rendering
- Email notification delivery

The application was continuously tested and debugged throughout development rather than being considered complete immediately after feature implementation.

22. Known Limitations

The current version is functionally complete for its currently implemented scope but remains in pre-deployment testing.

AI Support

The system currently does not include an AI assistant capable of handling common technical questions.

Notification Expansion

Additional notification indicators can be added to certain modules.

Online Presence

The current online status implementation is limited and does not represent a complete presence-tracking system.

Staff Email Accounts

IT Support and Technician password-change notifications require their designated staff email accounts and final email templates before complete end-to-end testing can be performed.

Scalability

The system has not yet undergone large-scale concurrent-user or load testing.

Production Security

Additional security testing and hardening should be completed before public deployment.

23. Future Improvements

AI Support Assistant

A future version could include an AI-powered support assistant.

AI Support Assistant

A potential future enhancement for the MollyTech Service Desk is the integration of an AI Support Assistant capable of handling basic technical concerns before escalating more complex issues to human support personnel.

Proposed Workflow

1. **User Inquiry** — The user describes a technical issue or support concern.
2. **AI Support Assistant** — The AI analyzes the request and provides troubleshooting guidance based on the available knowledge base.
3. **Basic Issue** — If the issue can be addressed through standard troubleshooting, the AI provides a suggested resolution.
4. **Complex Issue** — If the issue requires human intervention or cannot be resolved through the available guidance, the request is escalated.
5. **Ticket Creation** — The system creates or recommends creating a support ticket containing the relevant issue details.
6. **Human Support** — The ticket is routed to the appropriate IT Support staff member or Technician for further investigation.

This enhancement could reduce the workload associated with repetitive technical inquiries while allowing human support personnel to focus on more complex incidents. The AI assistant would serve as an initial support layer rather than replacing human technical support.

Scalability

Future versions should be optimized for:

- Larger user populations
- Increased ticket volume
- Concurrent messaging
- Larger databases
- Increased notification traffic

Performance

Potential improvements include:

- Database indexing
- Query optimization
- Caching
- Socket event optimization
- Frontend optimization
- Lazy loading

Security

Future security improvements include:

- Stronger validation
- Rate limiting
- Improved file validation
- Secure attachment handling
- Security auditing
- Production environment hardening
- Multi-factor authentication

Email Infrastructure

Future production deployment could include:

- Dedicated organizational email accounts
- Production SMTP provider
- Email delivery monitoring
- Email delivery retry handling
- Additional transactional email templates

24. Deployment Status

The current version is running in a local development environment and is undergoing final feature validation and pre-deployment testing.

Core functionality has been implemented and tested, including:

- Authentication
- Role-based access
- Ticket management
- Ticket assignment
- Real-time messaging
- Notifications
- Password management
- Password recovery
- Email-based security notifications for Administrator and User accounts

Before deployment, the system should undergo additional:

- Security testing
- Authentication testing
- Load testing
- Real-time testing
- File upload testing
- Database testing
- Email delivery testing
- Production configuration testing

A future deployment architecture is planned using **Vercel for the frontend** with an appropriate backend hosting environment and MongoDB deployment.

25. Developer Role

I personally developed the system from the frontend through the backend and database.

Responsibilities included:

- System architecture

- System design
- React development
- Vite configuration
- Node.js development
- Express development
- MongoDB database design
- Authentication
- Authorization
- Role-based access
- Ticket management
- Ticket assignment
- Real-time Socket.IO implementation
- Messaging system
- Notification system
- Email service integration
- Password management
- File handling
- Dashboard analytics
- UI/UX
- Testing
- Debugging
- System integration

The project was developed independently as a self-directed engineering project.

26. Developer Reflection

MollyTech Service Desk represents a significant step forward in my development approach compared with my earlier projects.

The goal was not simply to create another CRUD application. I intentionally challenged myself to design and develop a system that behaves more like a real-world IT service desk.

The project required consideration of:

- System architecture
- User workflows
- Support workflows
- Ticket lifecycle
- Role-based access
- Database relationships

- Real-time communication
- Notifications
- Authentication
- Account security
- Email integration
- UI/UX
- Testing
- Debugging
- Maintainability

One of the most important lessons from the project was that **software engineering is not simply about writing code.**

Planning, architecture, system design, testing, debugging, documentation, and continuous improvement are equally important.

The project also taught me the importance of understanding the entire system rather than only individual components. Because I developed the frontend, backend, database, authentication, email service, and real-time infrastructure, debugging became an exercise in understanding how the entire application communicates.

The project took approximately **two to three months to develop** and became one of the most technically challenging systems I have built so far.

Rather than considering the project finished simply because its major features work, I consider it an evolving engineering project that can continue to improve through security hardening, performance optimization, scalability testing, deployment, and future AI integration.

27. Screenshots

Figure 1 — Login Screen

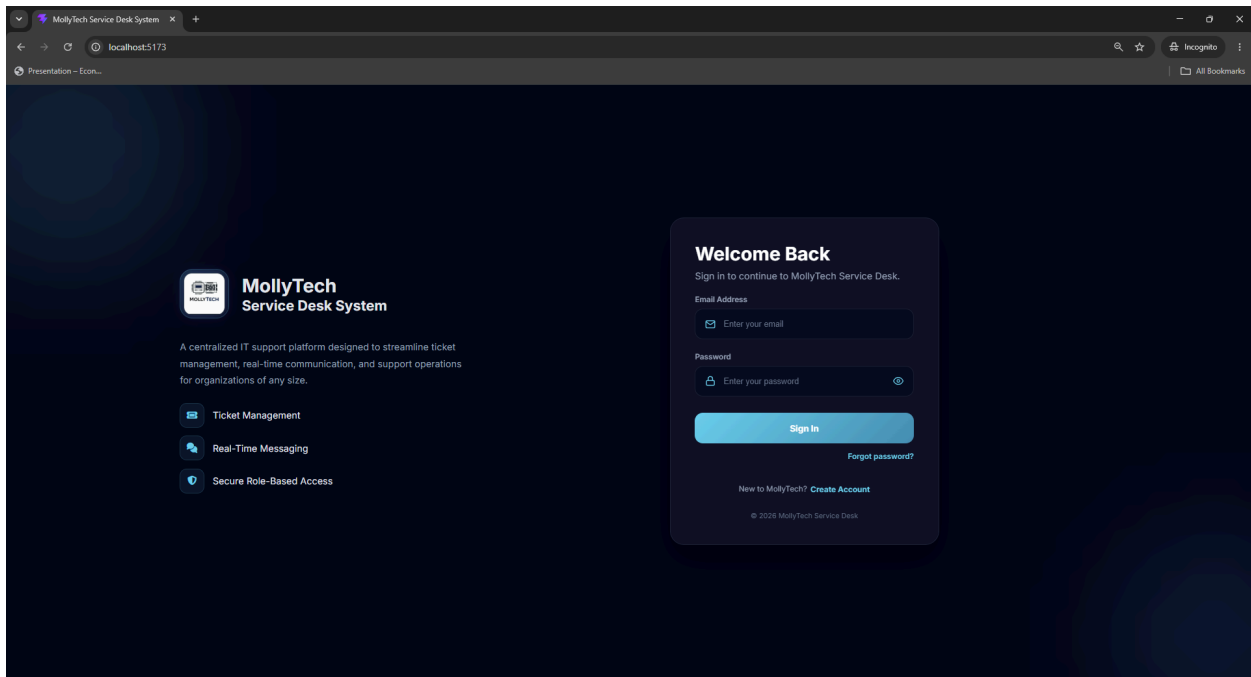
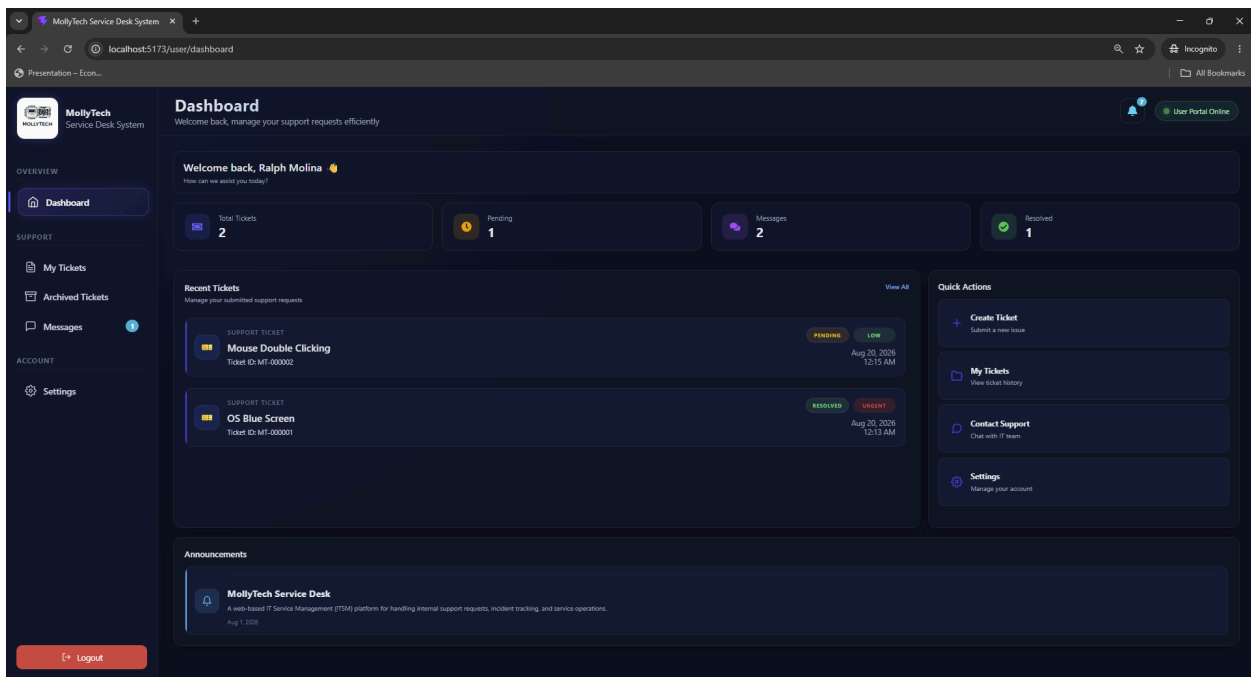


Figure 2 — User Dashboard



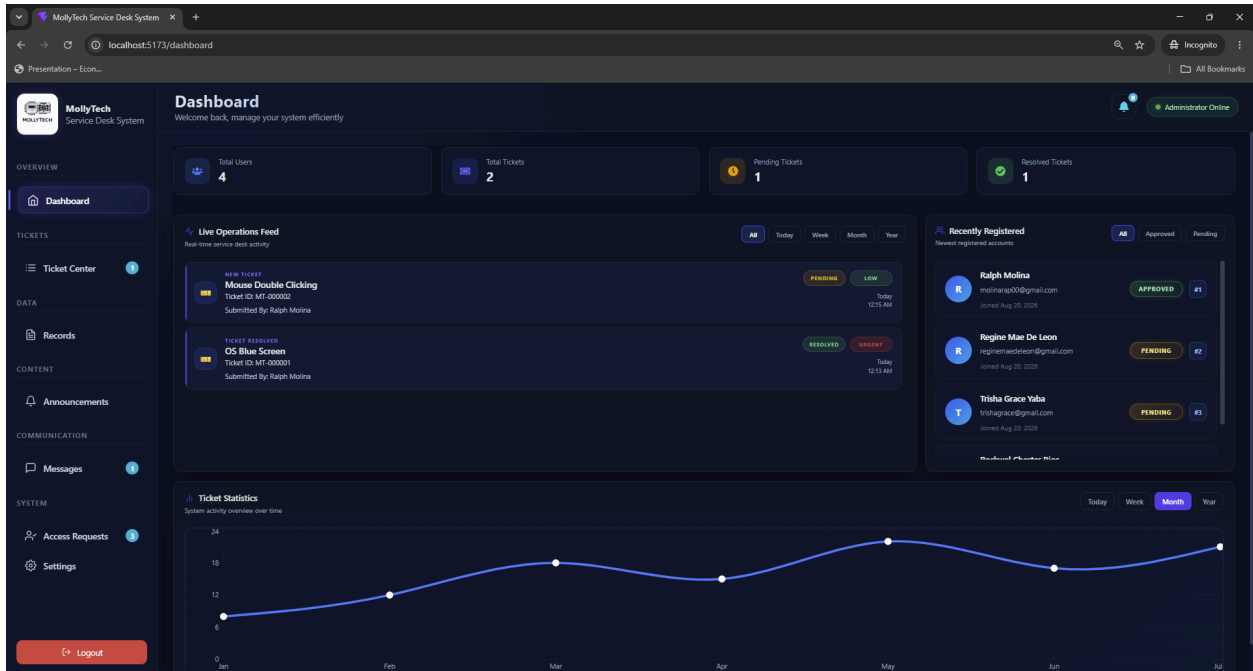


Figure 3 — Create Ticket

The screenshot shows the 'Create Support Ticket' form in the MollyTech Service Desk System. The form is titled 'Create Support Ticket' and includes a subtitle 'Submit an IT issue and our support team will review it as soon as possible.' The form is divided into several sections: 'Before submitting your request' (with instructions on how to submit a request), 'Request Details' (with fields for Subject, Category, and Priority), 'Contact Information' (with fields for Email and Phone Number), and 'Issue Description' (with a text area for the description and a list of example questions). The form is set against a dark background with a sidebar on the left.

Before submitting your request

- Use a clear and descriptive subject.
- Mention any error messages shown.
- Include the affected device or software.
- Describe the issue step-by-step.

Request Details

Subject *
Briefly describe your issue

Category *
Select Category

Priority
Low

Contact Information

Email *
molinarap00@gmail.com

Phone Number
+63 916 847 1713

Issue Description

Description *
Example:
• What were you doing?
• What happened?
• Any error messages?
• When did it start?

Figure 4 — My Tickets

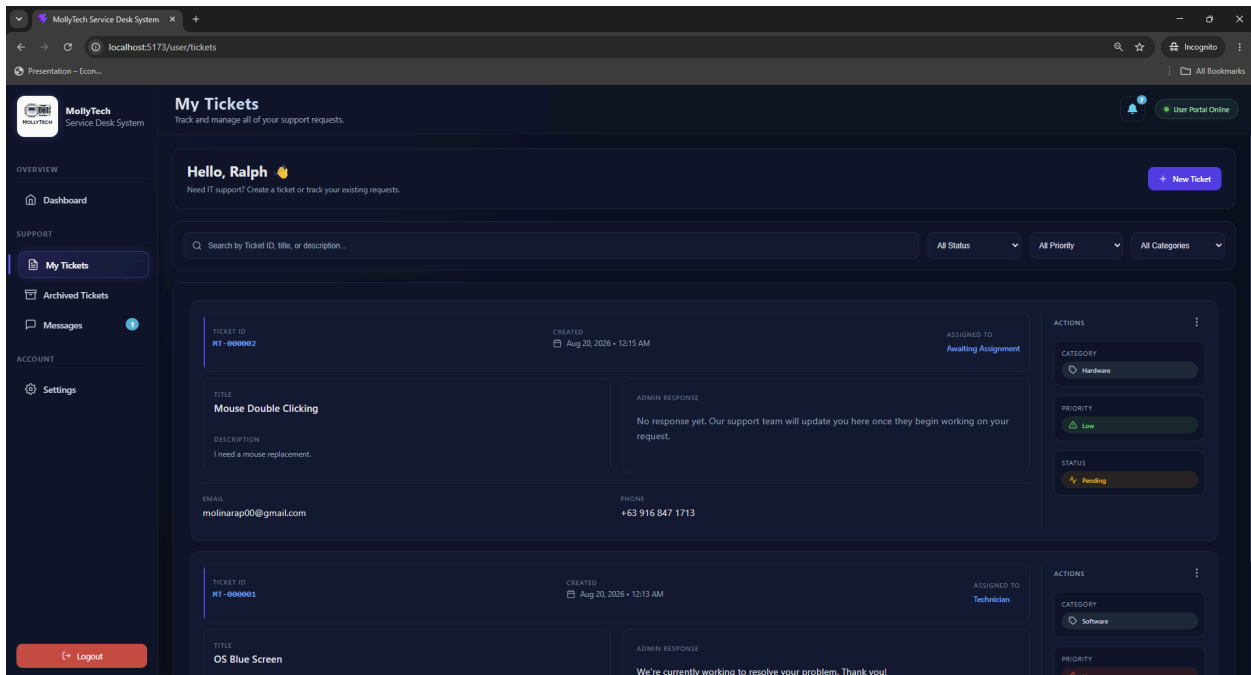


Figure 5 — Ticket Center

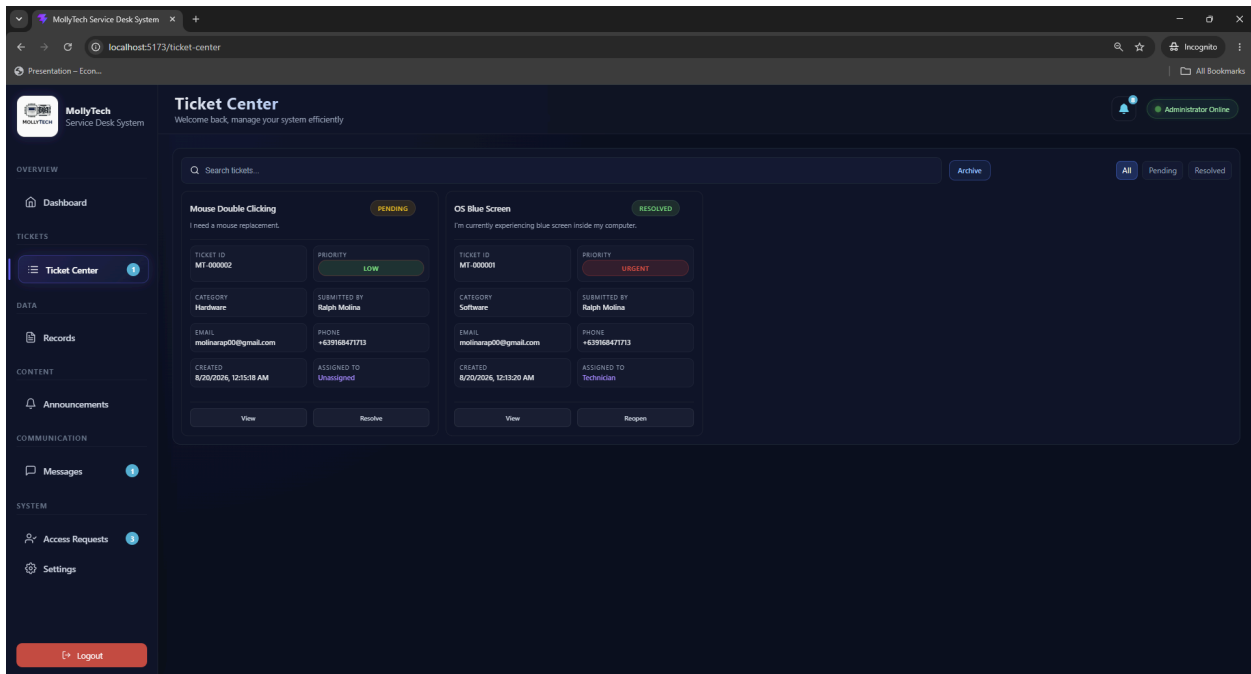


Figure 6 — Administrator Dashboard

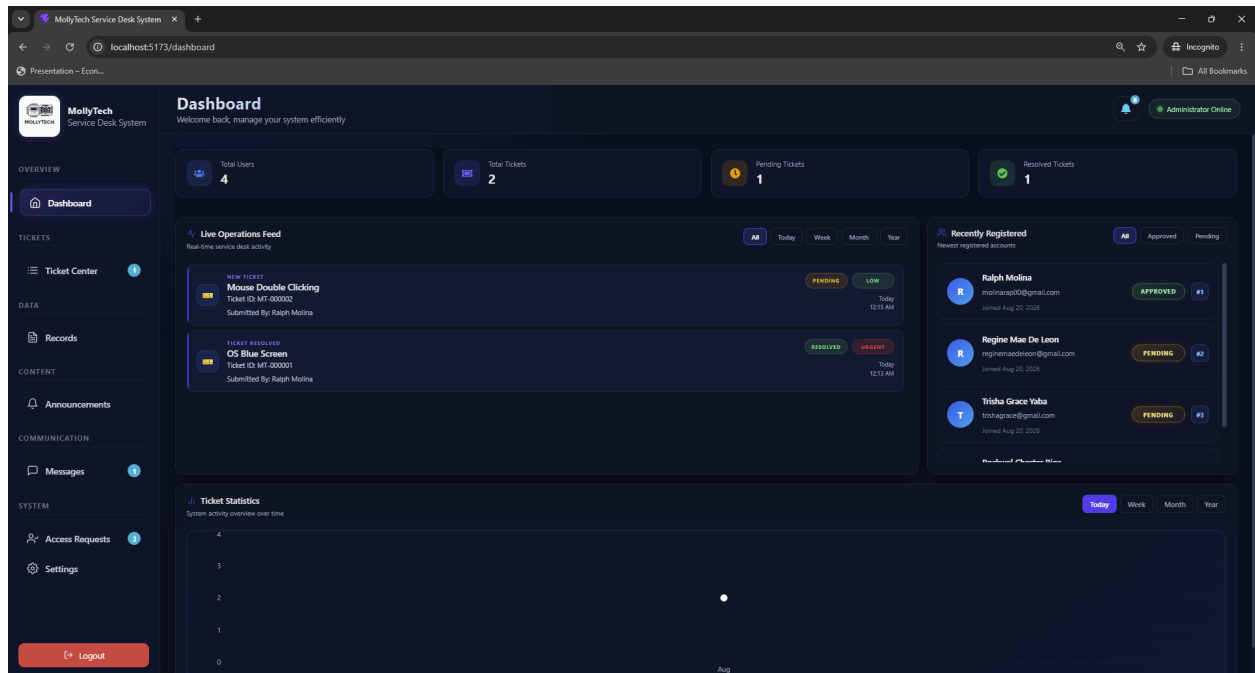


Figure 7 — Technician Dashboard

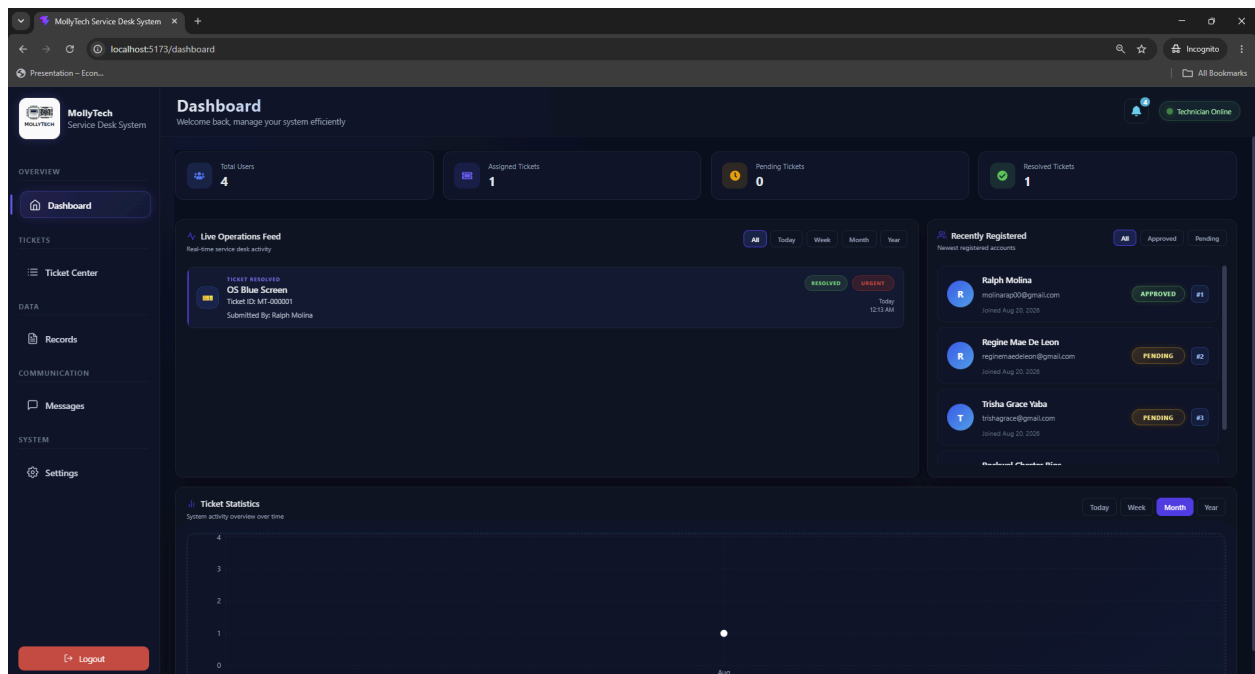


Figure 8 — Messages

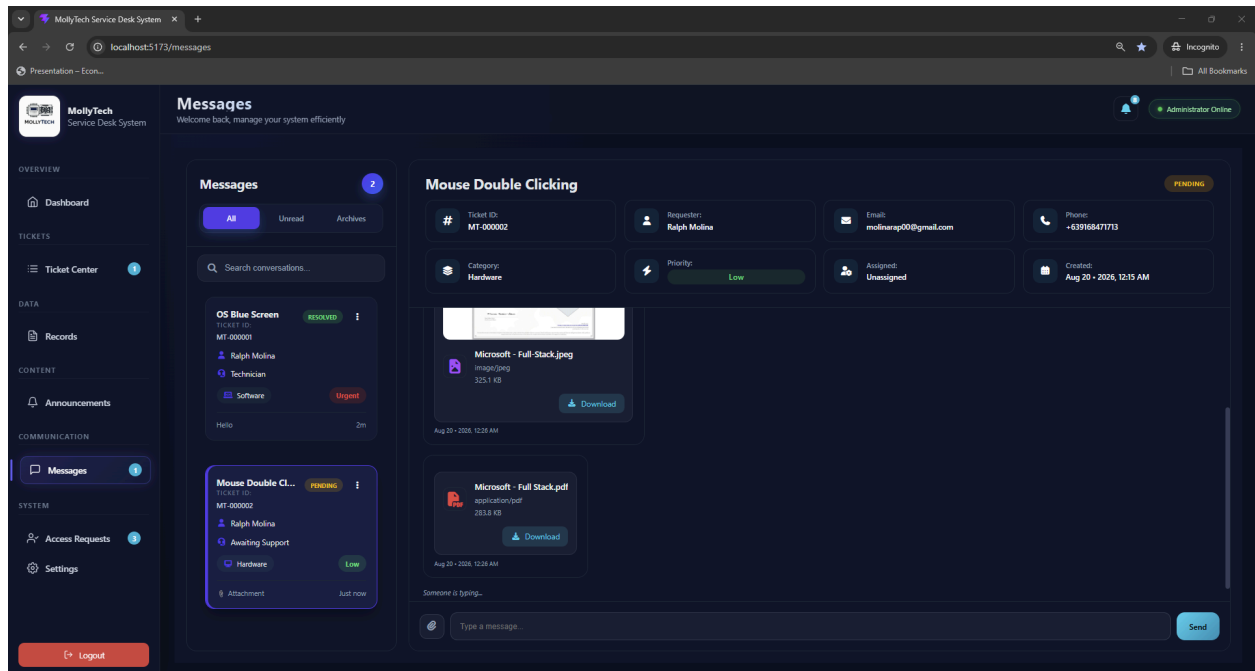


Figure 9 — Real-Time Conversation

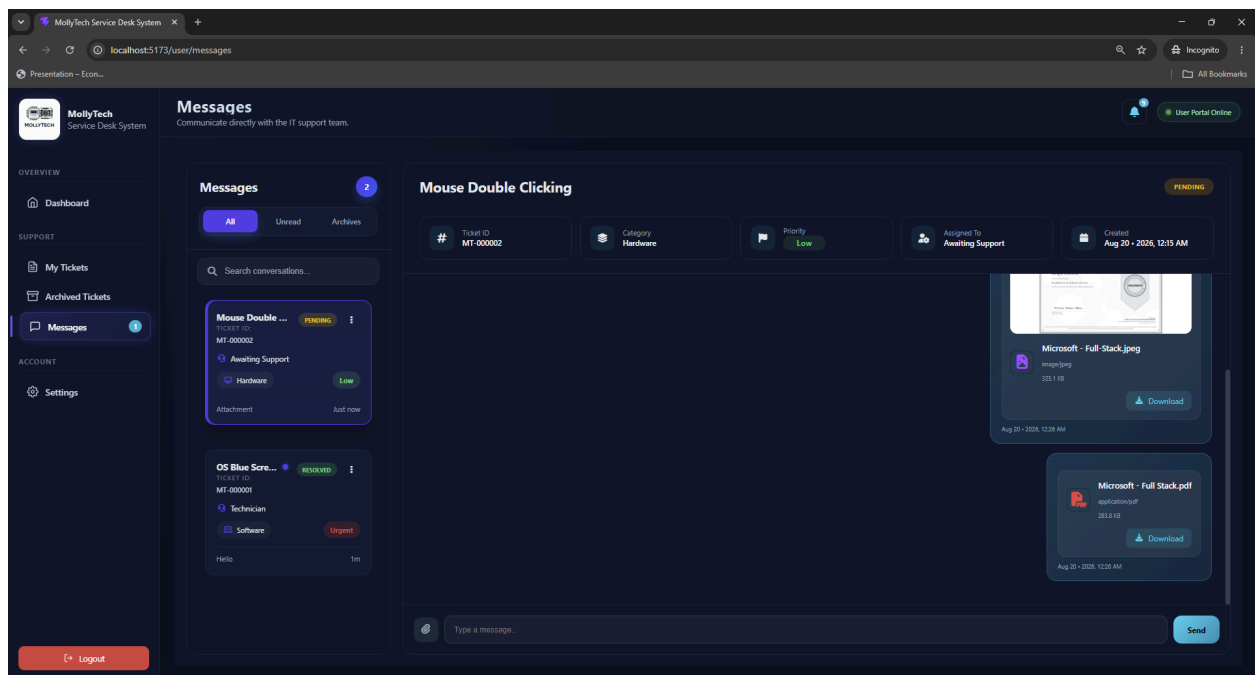


Figure 10 — Notifications

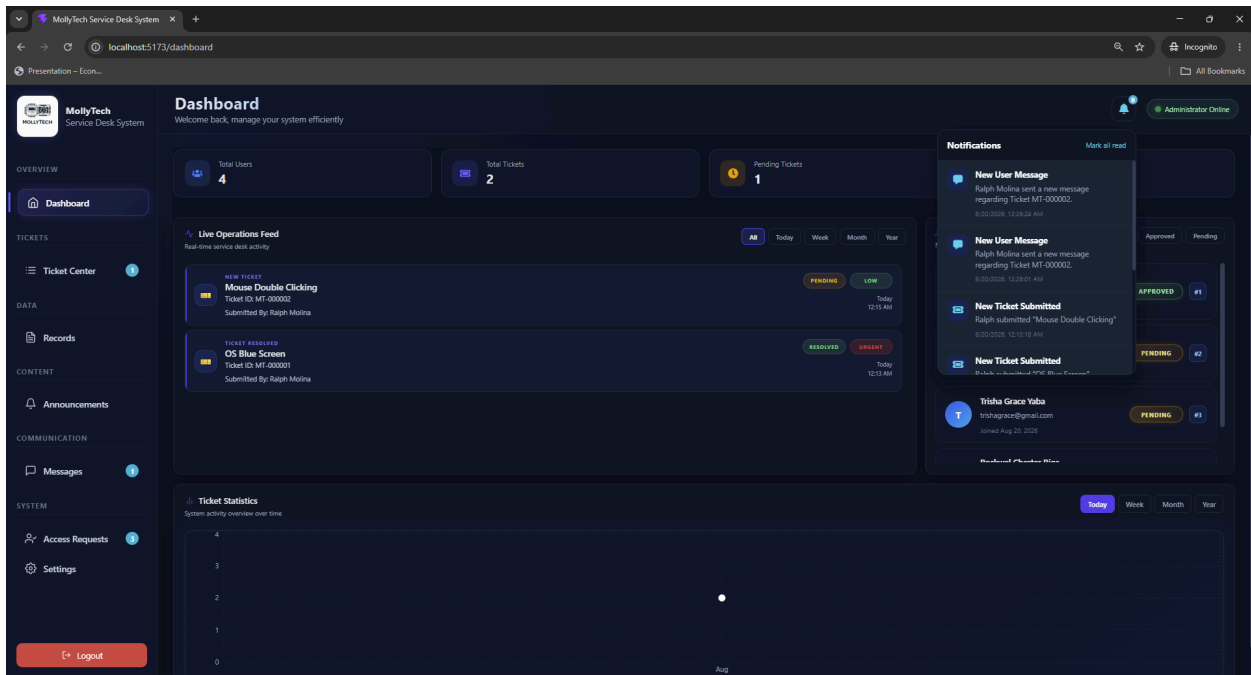


Figure 11 — Announcements

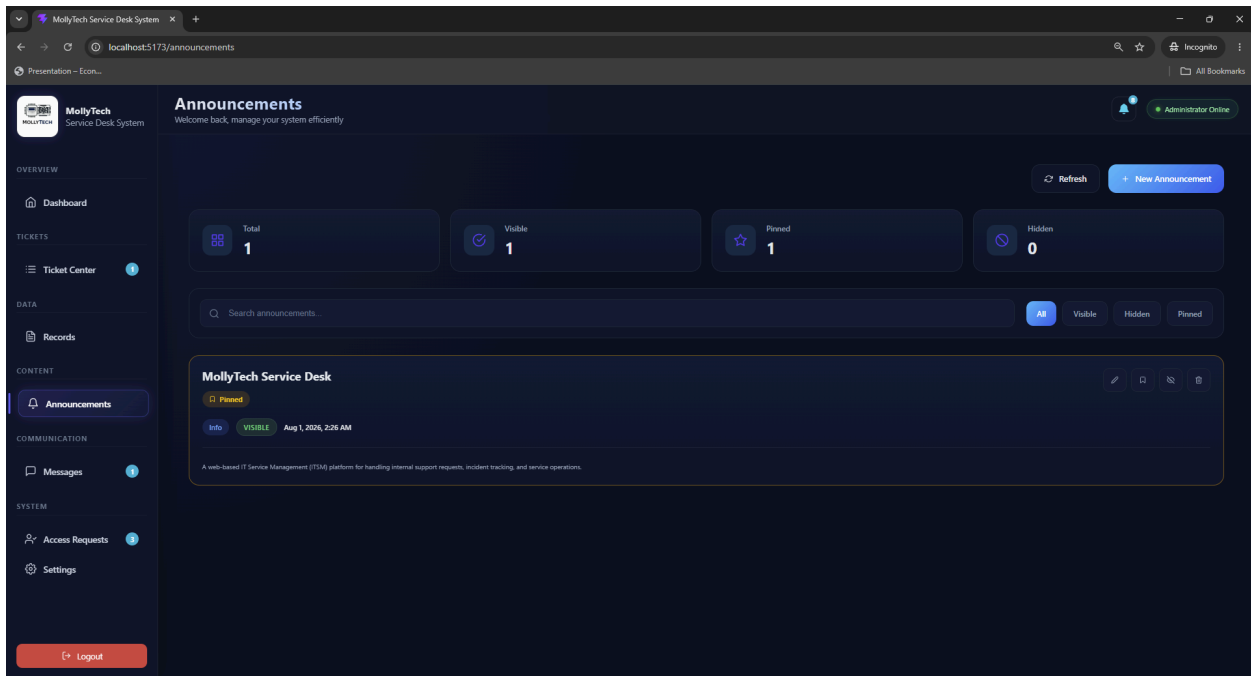


Figure 12 — Access Requests

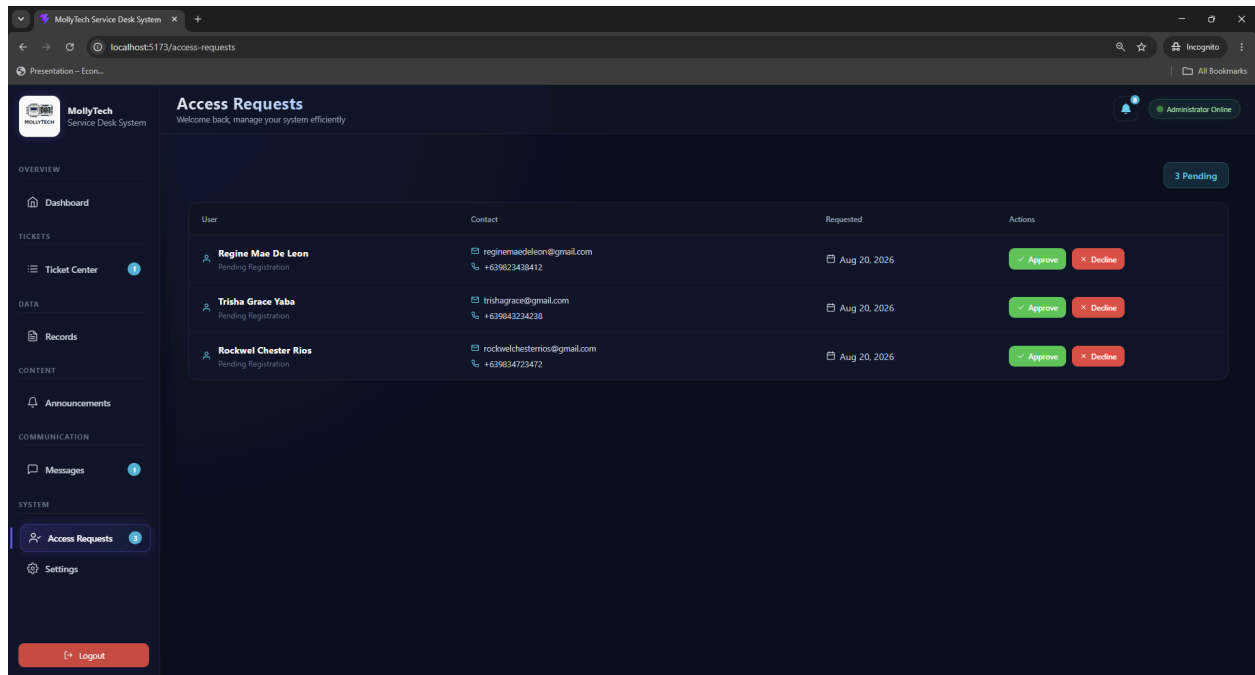


Figure 13 — Administrator Settings

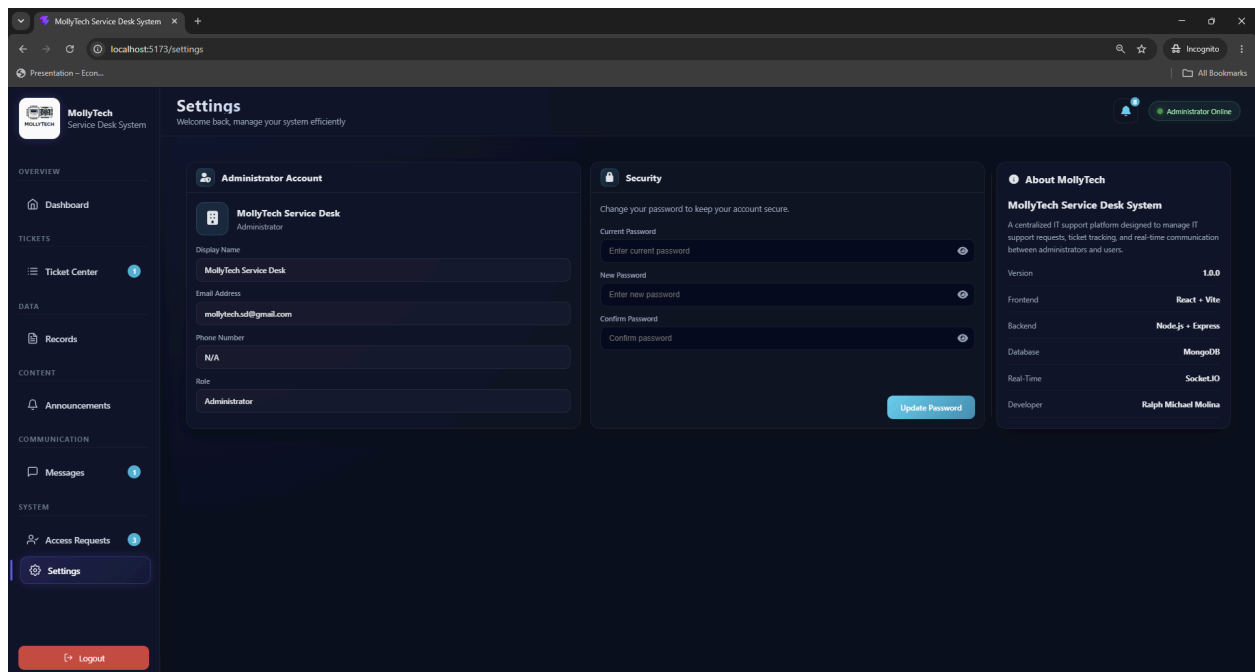
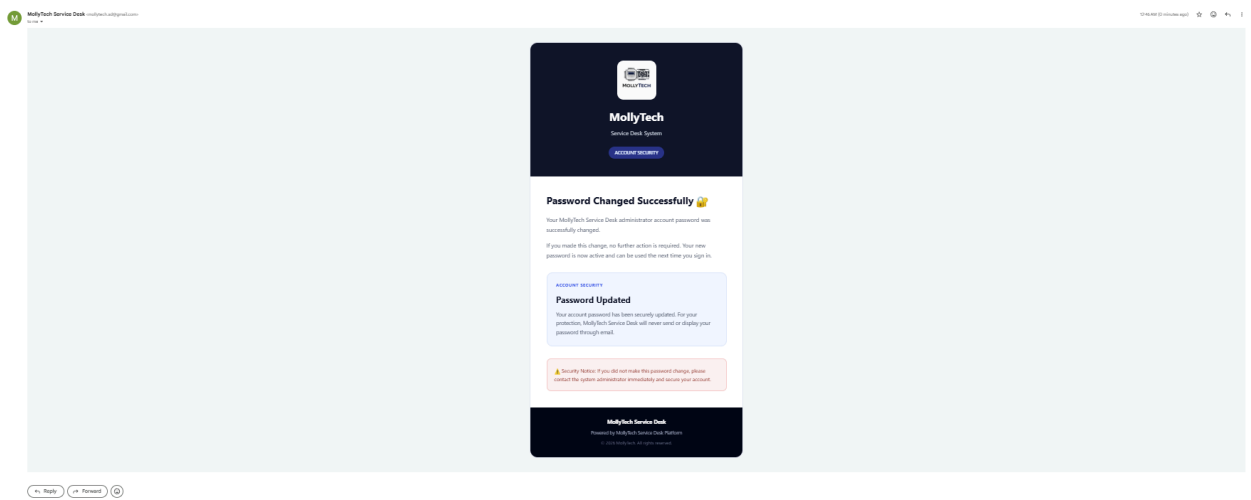
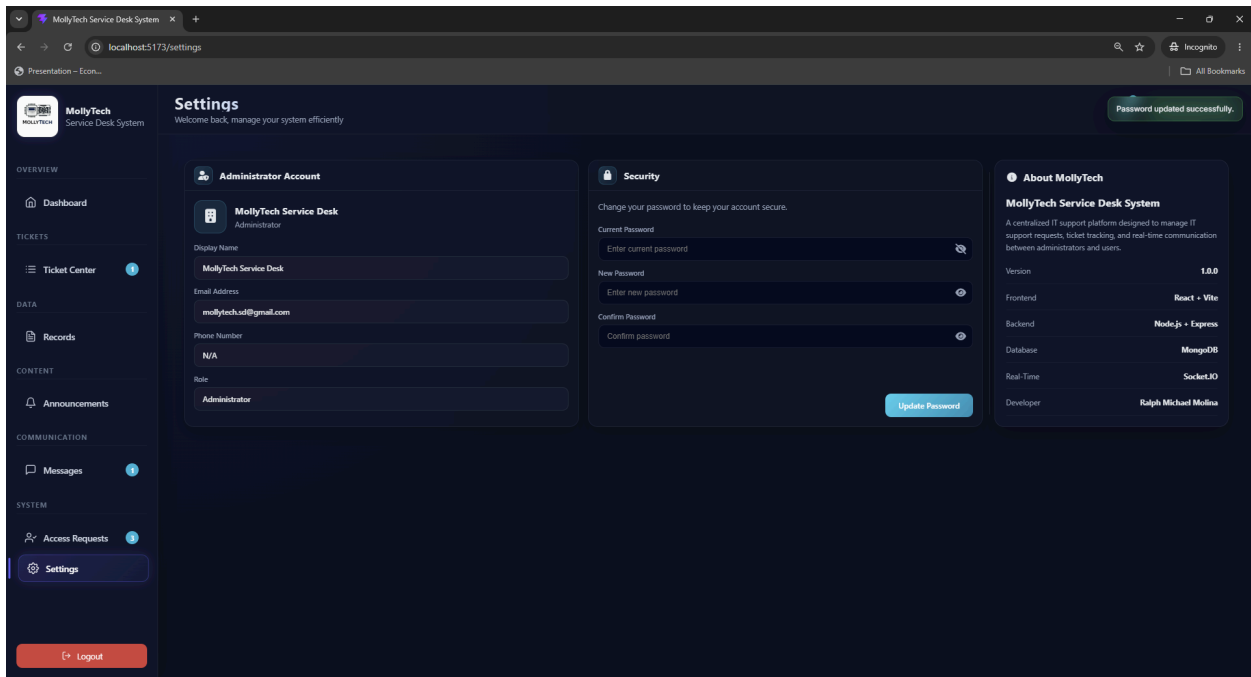


Figure 14 — Password Change Security Notification



28. Project Summary

Category	Details
Project	MollyTech Service Desk
Type	Full-Stack Software Engineering Project
Application	Web-Based IT Service Management System
Frontend	React + Vite
Backend	Node.js + Express
Database	MongoDB
Real-Time Communication	Socket.IO
Authentication	JWT + Email/Password Authentication
Roles	User, Administrator, IT Support, Technician
Messaging	Real-Time Ticket-Based Conversations
Notifications	Database-Backed Real-Time Notifications

Email Service	Nodemailer
Security Emails	Password-Change Notifications
File Handling	Image and File Attachments
Development	Self-Directed
Current Status	Functionally Complete / Pre-Deployment
Deployment	Local Development Environment
Developer Role	Software Engineer & Full-Stack Developer

29. Conclusion

MollyTech Service Desk demonstrates the development of a full-stack IT service management system designed around realistic support operations rather than a simple CRUD workflow.

The system integrates:

- Role-based access control
- Authentication
- Password management
- Password recovery
- Account security notifications
- Ticket management
- Ticket assignment
- Support workflows
- Real-time messaging

- Notifications
- File attachments
- MongoDB data management
- Dashboard analytics
- Administrative workflows
- User support functionality
- Transactional email integration

The project provided practical experience in designing and integrating multiple application layers into a single functioning system.

More importantly, the project was developed as an opportunity to **think and work from a software engineering perspective**—considering architecture, system behavior, maintainability, user experience, security, testing, and future scalability alongside implementation.

The current system provides a strong foundation for future development, including AI-assisted support, improved scalability, stronger security, performance optimization, dedicated production email infrastructure, and production deployment.